Implantação e Validação do MLP Quantizado em C (Semana 2)

> **Resumo da Semana**
>
> Esta segunda semana foi dedicada a construir e validar toda a infraestrutura base do projeto, saindo do alto nível em Python até o baixo nível em C. Começamos com o treino no scikit-learn, simulamos a quantização em Python e depois a inferência em C dividindo em três versões gradativas (float, int32 e int8). O teste final com as 10.000 imagens validou a integridade do modelo quantizado (acurácia de 97,28%) e revelou uma limitação natural de precisão do `float32` do C contra o `float64` do Python.

---

1. O que aprendi: Tipos de Dados e Limitações Práticas

Para construir um motor que funcionasse de forma limpa em sistemas embarcados (focando no futuro uso no Raspberry Pi), precisei descer o nível de abstração e lidar com o gerenciamento estrito de memória do C:

- **Simplificação Matemática:** Conforme discutido com o orientador, aprendi que podemos ignorar completamente a função Softmax na camada de saída. Para descobrir o dígito previsto, basta fazer uma varredura simples (`argmax`) e encontrar o maior valor no acumulador, o que elimina a necessidade de carregar a pesada biblioteca `<math.h>` .
- **A diferença entre os mundos:** O Python processa a álgebra linear em precisão dupla (`float64`). No C, ao utilizar `float` comum (32 bits) por eficiência, percebi que o arredondamento acumulado nas 64 multiplicações da camada oculta é suficiente para alterar levemente a decisão da rede em casos isolados.

---

2. A Estrutura do Código em C

Para não dar saltos no escuro, dividi a implantação em C em três motores independentes, além de um orquestrador para testá-los contra as imagens reais:

1. `01_inferencia_float.c` : Minha referência absoluta. Lê os pesos do Python ainda em ponto flutuante e executa a álgebra $XW + b$. O objetivo foi apenas provar que a lógica dos laços `for` estava mapeando as matrizes achatadas (1D) corretamente.
2. `02_inferencia_int32.c` : A primeira transição para números inteiros. Utiliza arrays `int32_t` para tudo. Ocupa bastante memória, mas me deu segurança para atestar que a matemática inteira convergia antes de eu comprimir os dados.

3. `03_inferencia_int8.c`: A versão definitiva e pronta para o hardware. Os pesos e ativações ocupam apenas 1 byte (`int8_t`). A conversão para `int32_t` acontece única e exclusivamente no exato pulso de clock da multiplicação, prevenindo *overflow* — exatamente o comportamento que vou delegar para as instruções SIMD/NEON do ARM na próxima semana.
4. `testar_acuracia.c`: O script de teste. Ele aloca o arquivo `X_te.bin` (contendo todas as 10.000 imagens de teste do MNIST) na RAM, injeta nos motores Float e Int8, tabula as discordâncias e imprime a matriz de confusão final.

---

3. Resultados e Provas de Convergência

Ao passar o lote de 10.000 imagens pelo validador em C, foi possível registrar com exatidão a manutenção da precisão:

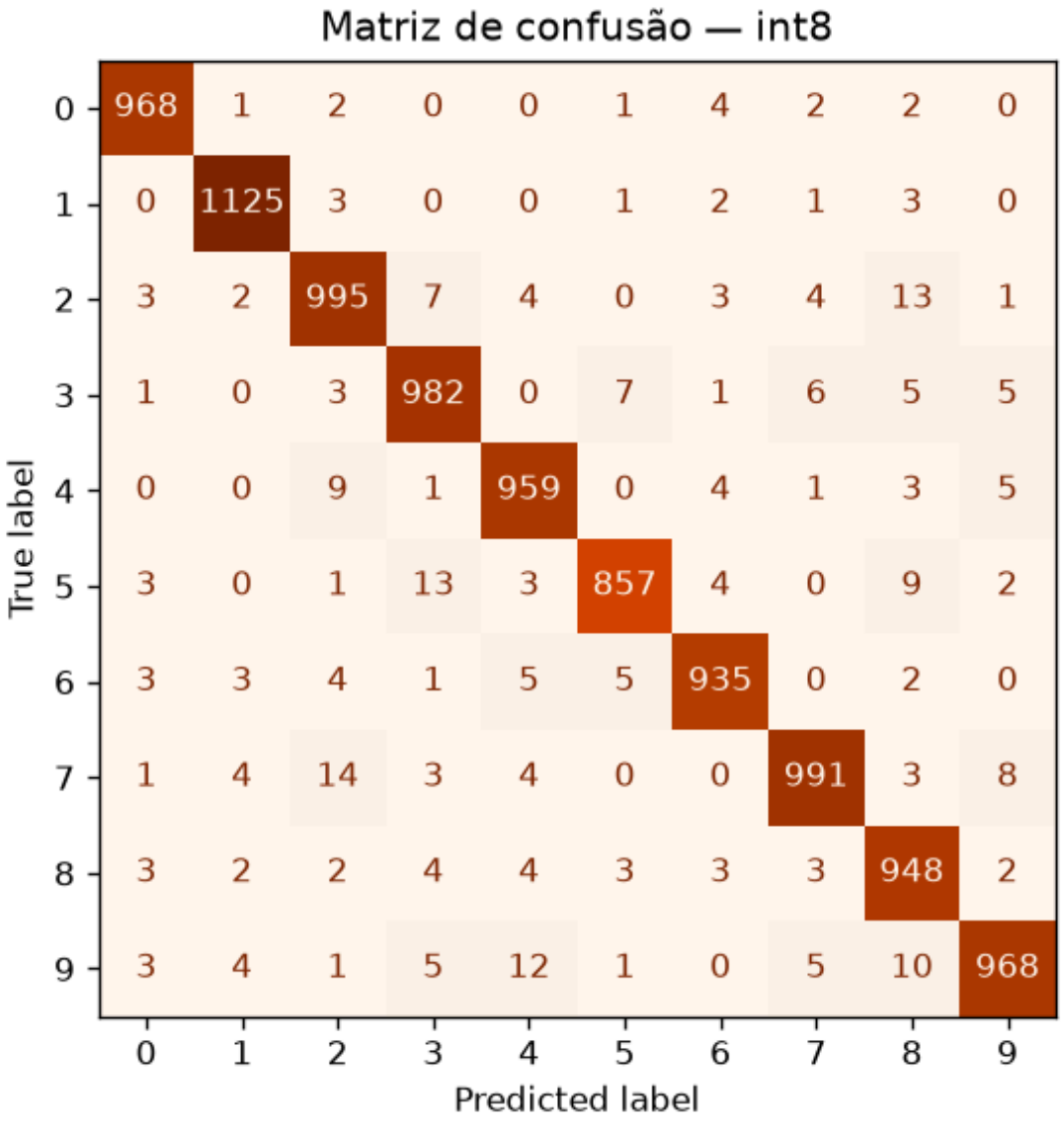
- **Acurácia em Python (float64):** 0,9708
- **Acurácia em C (float32):** 0,9704
- **Acurácia do Motor Otimizado C (int8):** 0,9728

O C(float32) divergiu do Int8 em exatas **50 amostras**. Esse pequeno desvio entre o baseline em Python (que alterou 47 decisões) provou a tese do orientador: a validação passo a passo isolou o "erro" de hardware (float32 vs float64) do erro da quantização, mostrando que o modelo Int8 se manteve robusto.

Comparação Visual (Python vs C)

Para atestar o funcionamento do código C, comparei a matriz de confusão gerada inicialmente na simulação do Python com a tabela cuspida pelo terminal Linux rodando o código C puro.

Matriz Teórica Simulada em Python:



Saída Real do Motor em C (Linux):

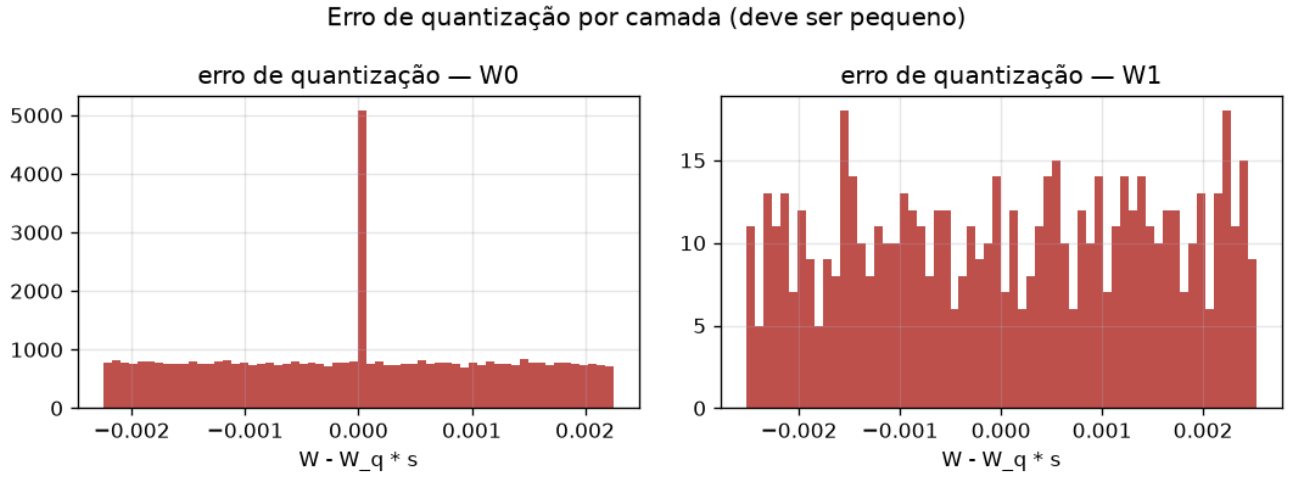
```
daviz@zangetsu:~/c$ ./testar_acuracia
Alocando memoria para 10.000 imagens...
Iniciando bateria de testes...

=======================================================
                RELATORIO DE VALIDACAO C
=======================================================
Total de Amostras Testadas: 10000
Acuracia Float32 (Referencia) : 0.9704
Acuracia Int8 (Otimizada)     : 0.9728
Decisoes Alteradas (Float vs Int8): 50 / 10000

==== MATRIZ DE CONFUSAO (INT8 em C) ====
   |    0    1    2    3    4    5    6    7    8    9
---+---------------------------------------------------
 0 | [968]    1    2    0    0    1    4    2    2    0
 1 |    0 [1125]    3    0    0    1    2    1    3    0
 2 |    3    2 [995]    7    4    0    3    4   13    1
 3 |    1    0    3 [982]    0    7    1    6    5    5
 4 |    0    0    9    1 [959]    0    4    1    3    5
 5 |    3    0    1   13    3 [857]    4    0    9    2
 6 |    3    3    4    1    5    5 [935]    0    2    0
 7 |    1    4   14    3    4    0    0 [991]    3    8
 8 |    3    2    2    4    4    3    3    3 [948]    2
 9 |    3    4    1    5   12    1    0    5   10 [968]
=======================================================
daviz@zangetsu:~/c$
```

Os placares por dígito bateram perfeitamente (ex.: 1125 acertos no dígito 1, 991 no dígito 7). Isso atesta que o controle dos erros de quantização funcionou como esperado:

Histograma do Erro de Quantização (Python):



A ausência de caudas longas no histograma de erro acima permitiu que o modelo em C mantesse o desempenho estável mesmo após o agressivo esmagamento das matrizes.

4. Próximos Passos (Semana 2)

- Transportar os códigos limpos em C e o cabeçalho `pesos.h` para o ambiente do Raspberry Pi 4.
- Adaptar o código `inferencia_int8.c` para incluir as diretivas `<time.h>` e medir o tempo (em milissegundos) de execução da versão escalar no processador Cortex-A72.
- Iniciar os estudos sobre as instruções intrínsecas NEON (ARMv8) para vetorizar o laço de produto matriz-vetor e medir o *speedup* real de hardware.